PROGRAMMING FOR PROBLEM SOLVING

UNIT 2 Problem Decomposition, Functions and Pointers

PREMANAND S

Assistant Professor

Department of Computational Intelligence

SRM Institute of Science and Technology

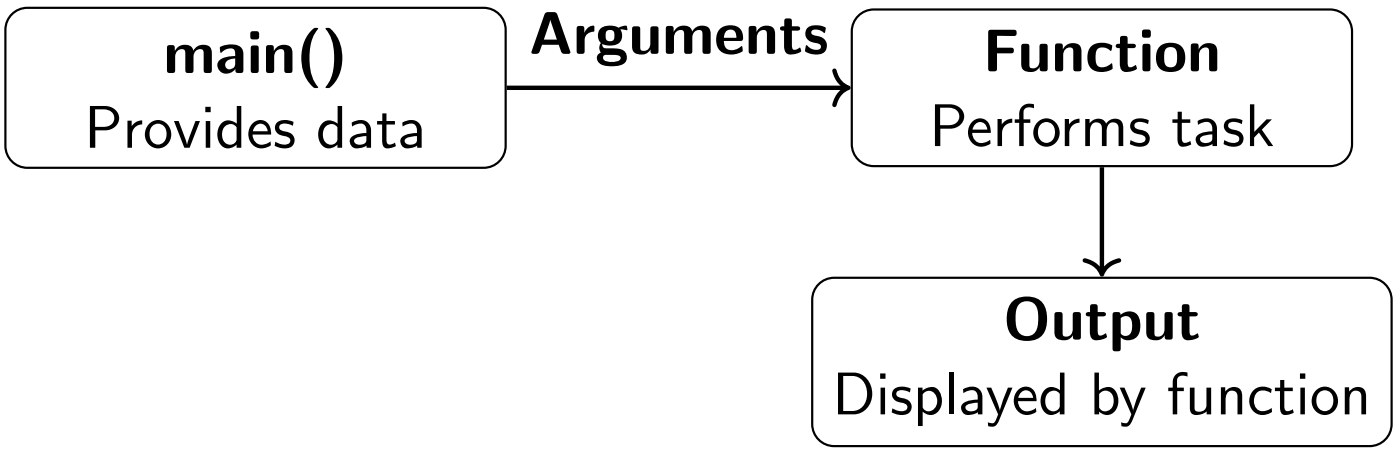
Kattankulathur Campus

First Year Engineering

Type 2: Arguments + No Return Value

What Does It Mean?

- The function **receives data** from the calling function.
- The function performs a specific task using that data.
- The function **does not return a value**.



Syntax

```
void functionName(data_type parameter);
```

Type 2 — Problem 1

Problem

Create a function `displayNumber()` that accepts a number as an argument and displays the number.
The function should:

- accept **one argument**
- return **no value**

Type 2 — Solution 1

Example

```
#include <stdio.h>
void displayNumber(int n);
int main()
{
 displayNumber(25);
 return 0;
}
void displayNumber(int n)
{
 printf("Number = %d", n);
}
```

Type 2 — Problem 2

Problem

Create a function `displaySquare()` that accepts a number as an argument and displays its square.

- Accept **one argument**
- Calculate the square inside the function
- Display the result
- Return **no value**

Example

Input: `6`

Output: `Square = 36`

Type 2 — Solution 2

Solution

```
#include <stdio.h>
void displaySquare(int n);
int main()
{
 displaySquare(6);
 return 0;
}
void displaySquare(int n)
{
 int square;
 square = n * n;
 printf("Square = %d", square);
}
```

Type 2 — Problem 3

Scenario

An electricity board wants to calculate a customer's bill based on the number of units consumed.
Create a function `calculateBill()` that accepts the number of units as an argument and **displays** the electricity bill.

Billing Rules

- First 100 units $\rightarrow$ Rs. 2 per unit
- Next 100 units $\rightarrow$ Rs. 3 per unit
- Above 200 units $\rightarrow$ Rs. 5 per unit

Example

If the customer consumes **250 units**, calculate and display the bill using the above slabs.

Type 2 — Solution 3

C Program

```
#include <stdio.h>
void calculateBill(int units);
int main()
{
 int units;
 printf("Enter units consumed: ");
 scanf("%d", &units);
 calculateBill(units);
 return 0;
}
```

Type 2 — Solution 3

C Program

```
void calculateBill(int units)
{
 float bill;
 if (units <= 100)
 bill = units * 2;
 else if (units <= 200)
 bill = (100 * 2) + ((units - 100) * 3);
 else
 bill = (100 * 2) + (100 * 3)
 + ((units - 200) * 5);
 printf("Electricity Bill = Rs. %.2f", bill);
}
```

Type 2 — Practice Question 1

Problem

Create a function `displayCube()` that accepts a number as an argument and displays its cube.
For example:

Input: `4`
Output: `Cube = 64`

Practice Question 1 — Solution

Solution

```
#include <stdio.h>
void displayCube(int n);
int main()
{
 int number;
 printf("Enter a number: ");
 scanf("%d", &number);
 displayCube(number);
 return 0;
}
```

Practice Question 1 — Solution

Solution

```
void displayCube(int n)
{
 int cube;
 cube = n * n * n;
 printf("Cube = %d", cube);
}
```

Type 2 — Practice Question 2

Problem

Create a function `displayResult()` that accepts the marks of three subjects as arguments and displays:

- Total marks
- Average marks

For example:

Marks: 80, 75, 90

Expected Output

```
Total = 245
Average = 81.67
```

Practice Question 2 — Solution

Solution

```
#include <stdio.h>
void displayResult(int m1, int m2, int m3);
int main()
{
 int mark1, mark2, mark3;
 printf("Enter marks in three subjects: ");
 scanf("%d %d %d", &mark1, &mark2, &mark3);
 displayResult(mark1, mark2, mark3);
 return 0;
}
```

Practice Question 2 — Solution

Solution

```
void displayResult(int m1, int m2, int m3)
{
 int total;
 float average;
 total = m1 + m2 + m3;
 average = total / 3.0;
 printf("Total = %d\n", total);
 printf("Average = %.2f", average);
}
```

Type 2 — Practice Question 3

Problem

Create a function `classifyNumber()` that accepts one integer as an argument and displays whether the number is:

- Positive
- Negative
- Zero

Examples

```
Input:  25   →   Positive
Input:  -8   →   Negative
   Input:  0   →   Zero
```

Practice Question 3 — Solution

Solution

```
#include <stdio.h>
void classifyNumber(int n);
int main()
{
 int number;
 printf("Enter a number: ");
 scanf("%d", &number);
 classifyNumber(number);
 return 0;
}
```

Practice Question 3 — Solution

Solution

```
void classifyNumber(int n)
{
 if (n > 0)
 {
  printf("Positive");
 }
 else if (n < 0)
 {
  printf("Negative");
 }
 else
 {
  printf("Zero");
 }
}
```

Type 2 — Practice Question 4

Scenario

Create a function `calculateBill()` that accepts the number of electricity units consumed and displays the bill.
Use the following billing rules:

- First 100 units → Rs. 2 per unit
- Next 100 units → Rs. 3 per unit
- Above 200 units → Rs. 5 per unit

Example

Units consumed: 250
Calculate and display the electricity bill.

Practice Question 4 — Solution

Solution

```
#include <stdio.h>
void calculateBill(int units);
int main()
{
 int units;
 printf("Enter units consumed: ");
 scanf("%d", &units);
 calculateBill(units);
 return 0;
}
```

Practice Question 4 — Solution

Solution

```
void calculateBill(int units)
{
 float bill;
 if (units <= 100)
 {
  bill = units * 2;
 }
 else if (units <= 200)
 {
  bill = (100 * 2) + ((units - 100) * 3); }
 else
 {
  bill = (100 * 2) + (100 * 3)   + ((units - 200) * 5); }
 printf("Electricity Bill = Rs. %.2f", bill);
}
```

Type 2 — Practice Question 5

Scenario

Create a function `classifyTriangle()` that accepts the three sides of a triangle as arguments and displays the type of triangle.

Classification

- Three equal sides → Equilateral
- Two equal sides → Isosceles
- All different sides → Scalene

Practice Question 5 — Solution

Solution

```
#include <stdio.h>
void classifyTriangle(int a, int b, int c);
int main()
{
 int side1, side2, side3;
 printf("Enter three sides: ");
 scanf("%d %d %d", &side1, &side2, &side3);
 classifyTriangle(side1, side2, side3);
 return 0;
}
```

Practice Question 5 — Solution

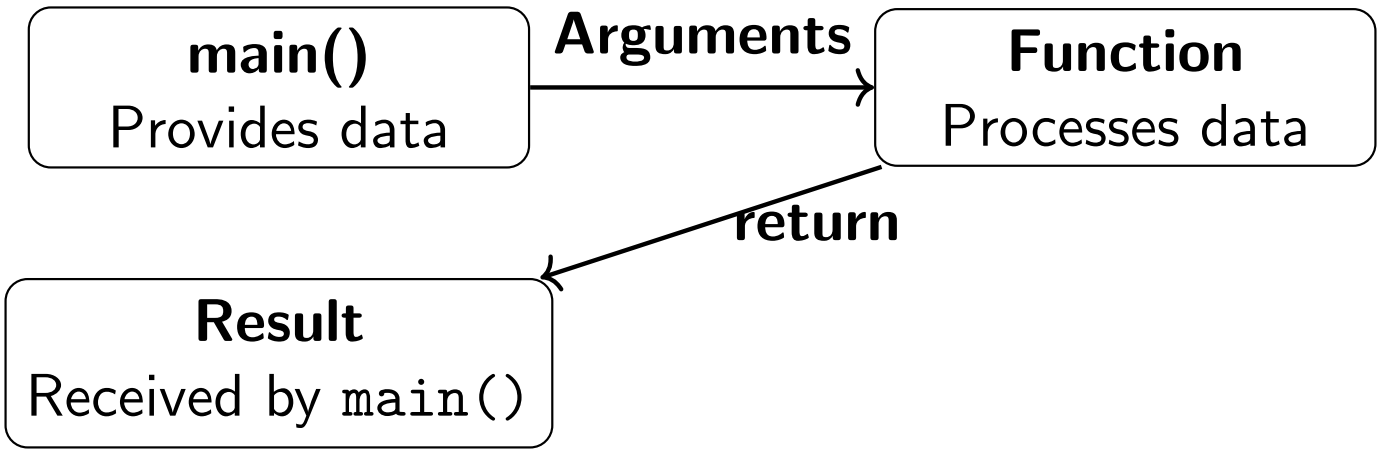
Solution

```
void classifyTriangle(int a, int b, int c)
{
 if (a == b && b == c)
 {
  printf("Equilateral Triangle");
 }
 else if (a == b || b == c || a == c)
 {
  printf("Isosceles Triangle");
 }
 else
 {
  printf("Scalene Triangle");
 }
}
```

Type 3 — Arguments + Return Value

What Does It Mean?

- The function **receives data** through arguments.
- The function performs a specific task using that data.
- The function **returns a result** to the calling function.



Syntax

```
return_type functionName(data_type parameter);
```

Type 3 — Simple Example

Problem

Create a function `add()` that accepts two numbers, calculates their sum, and **returns the result**.

- Accept **two arguments**
- Calculate the sum
- Return the sum to `main()`

Example

Input: `10, 20`

Function: `add(10, 20)`

Returned Result: 30

Type 3 — Solution

Solution

```
#include <stdio.h>
int add(int a, int b);
int main()
{
 int result;
 result = add(10, 20);
 printf("Sum = %d", result);
 return 0;
}
int add(int a, int b)
{
 int sum;
 sum = a + b;
 return sum;
}
```

Type 3 — Example

Problem

Create a function `largest()` that accepts two integers as arguments and **returns the larger number**.

- Accept two arguments
- Compare the two numbers inside the function
- Return the larger number
- Display the returned result in `main()`

Example

Input: `25, 40`

Returned Result: `40`

Type 3 — Solution

Solution

```
#include <stdio.h>
int largest(int a, int b);
int main()
{
 int num1, num2, result;
 printf("Enter two numbers: ");
 scanf("%d %d", &num1, &num2);
 result = largest(num1, num2);
 printf("Largest = %d", result);
 return 0;
}
```

Type 3 — Solution

Solution

```
int largest(int a, int b)
{
 if (a > b)
 return a;
 else
 return b;
}
```

Type 3 — Example

Scenario

A college system needs to determine a student's grade based on their marks.
Create a function `findGrade()` that accepts the student's marks as an argument and **returns the grade**.

Grading Rules

- Marks $\geq 90 \rightarrow$ Grade A
- Marks $\geq 75 \rightarrow$ Grade B
- Marks $\geq 60 \rightarrow$ Grade C
- Marks $\geq 50 \rightarrow$ Grade D
- Marks $< 50 \rightarrow$ Grade F

Type 3 — Solution

Solution

```
#include <stdio.h>
char findGrade(int marks);
int main()
{
 int marks;
 char grade;
 printf("Enter marks: ");
 scanf("%d", &marks);
 grade = findGrade(marks);
 printf("Grade = %c", grade);
 return 0;
}
```

Type 3 — Solution

Solution

```
char findGrade(int marks)
{
 if (marks >= 90)
 return 'A';
 else if (marks >= 75)
 return 'B';
 else if (marks >= 60)
 return 'C';
 else if (marks >= 50)
 return 'D';
 else
 return 'F';
}
```

Type 3 — Practice Question 1

Problem

Create a function `multiply()` that accepts two integers as arguments and **returns their product**.

For example:

Input: `6, 7`

Output: `Product = 42`

Practice Question 1 — Solution

Solution

```
#include <stdio.h>
int multiply(int a, int b);
int main()
{
 int num1, num2, result;
 printf("Enter two numbers: ");
 scanf("%d %d", &num1, &num2);
 result = multiply(num1, num2);
 printf("Product = %d", result);
 return 0;
}
```

Practice Question 1 — Solution

Solution

```
int multiply(int a, int b)
{
 int product;
 product = a * b;
 return product;
}
```

Type 3 — Practice Question 2

Problem

Create a function `factorial()` that accepts an integer as an argument and **returns its factorial**.
For example:

Input: `5`
Output: `Factorial = 120`

Type 3 — Practice Question 2 — Solution

Solution

```
#include <stdio.h>
int factorial(int n);
int main()
{
 int number;
 int result;
 printf("Enter a number: ");
 scanf("%d", &number);
 result = factorial(number);
 printf("Factorial = %d", result);
 return 0;
}
```

Type 3 — Practice Question 2 — Solution

Solution

```
int factorial(int n)
{
 int i;
 int fact = 1;
 for (i = 1; i <= n; i++)
 {
  fact = fact * i;
 }
 return fact;
}
```

Type 3 — Practice Question 3

Problem

Create a function `largest()` that accepts three integers as arguments and **returns the largest number**.
For example:

Input: `25, 48, 31`
Output: `Largest = 48`

Practice Question 3 — Solution

Solution

```
#include <stdio.h>
int largest(int a, int b, int c);
int main()
{
 int num1, num2, num3;
 int result;
 printf("Enter three numbers: ");
 scanf("%d %d %d", &num1, &num2, &num3);
 result = largest(num1, num2, num3);
 printf("Largest = %d", result);
 return 0;
}
```

Practice Question 3 — Solution

Solution

```
int largest(int a, int b, int c)
{
 if (a >= b && a >= c)
 {
  return a;
 }
 else if (b >= a && b >= c)
 {
  return b;
 }
 else
 {
  return c;
 }
}
```

Type 3 — Practice Question 4

Scenario

A college wants to automatically classify a student's result based on three subject marks.
Create a function `getResult()` that accepts the marks of three subjects and **returns** the result category.

Practice Question 4 — Solution

Solution

```
#include <stdio.h>
char getResult(int m1, int m2, int m3);
int main()
{
 int mark1, mark2, mark3;
 char result;
 printf("Enter marks in three subjects: ");
 scanf("%d %d %d", &mark1, &mark2, &mark3);
 result = getResult(mark1, mark2, mark3);
 printf("Result = %c", result);
 return 0;
}
```

Practice Question 4 — Solution

Solution

```
char getResult(int m1, int m2, int m3)
{
 float average;
 if (m1 < 40 || m2 < 40 || m3 < 40)
 {
  return 'F';
 }
 average = (m1 + m2 + m3) / 3.0;
```

Practice Question 4 — Solution

Solution

```
    if (average >= 75)
     return 'D';
    else if (average >= 60)
     return '1';
    else if (average >= 50)
     return '2';
    else
     return 'P';
  }
```

Type 3 — Practice Question 5

Scenario

Create a function `calculate()` that accepts two numbers and an operator as arguments and **returns the result**.

Supported Operators

`+` `-` `*` `/`

Example

Input: `20, 5, +`

Output: `Result = 25`

Practice Question 5 — Solution

Solution

```
#include <stdio.h>
float calculate(float a, float b, char op);
int main()
{
 float num1, num2, result;
 char operator;
 printf("Enter two numbers: ");
 scanf("%f %f", &num1, &num2);
 printf("Enter operator (+, -, *, /): ");
 scanf(" %c", &operator);
 result = calculate(num1, num2, operator);
 printf("Result = %.2f", result);
 return 0;
}
```

Practice Question 5 — Solution

Solution

```
float calculate(float a, float b, char op)
{
 if (op == '+')
  return a + b;
 else if (op == '-')
  return a - b;
 else if (op == '*')
  return a * b;
 else if (op == '/')
 {
  if (b != 0)
   return a / b;
  else
   return 0;
 }
 else
```

Practice Question 5 — Solution

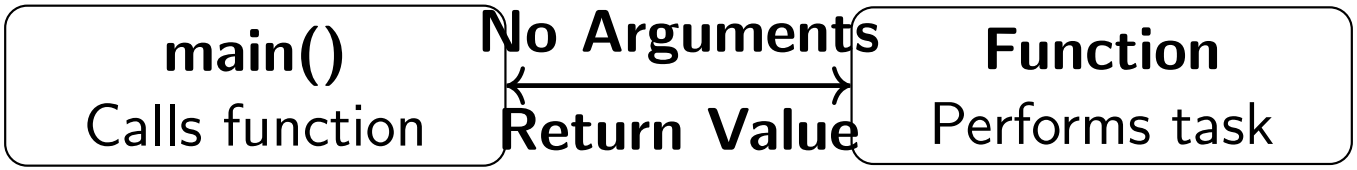
Solution

```
float calculate(float a, float b, char op)
{
 else if (op == '/')
 {
  if (b != 0)
   return a / b;
  else
   return 0;
 }
 else
  return 0;
}
```

Function with No Arguments and Return Value

What Does It Mean?

- The function **does not receive any data** from `main()`.
- The function performs a task internally.
- The function **returns a result** to the calling function.



Syntax

```
return_type functionName(void);
```

Type 4 — Example

Problem

Create a function `getNumber()` that:

- takes **no arguments**
- reads an integer from the user
- returns the number to `main()`

Example

User enters: 25

Function returns: 25

`main()` displays: `Number = 25`

Type 4 — Solution

Solution

```
#include <stdio.h>
int getNumber(void);
int main()
{
 int number;
 number = getNumber();
 printf("Number = %d", number);
 return 0;
}
int getNumber(void)
{
 int n;
 printf("Enter a number: ");
 scanf("%d", &n);
 return n;
}
```

Type 4 — Example

Problem

Create a function `generateNumber()` that:

- takes **no arguments**
- generates a random number between 1 and 100
- returns the generated number to `main()`

Example

Function call: `generateNumber()`

Possible returned value: `57`

`main()` displays: `Generated Number = 57`

Type 4 — Solution

Solution

```
#include <stdio.h>
int generateNumber(void);
int main()
{
 int number;
 number = generateNumber();
 printf("Number = %d", number);
 return 0;
}
int generateNumber(void)
{
 int n;
 printf("Enter a number: ");
 scanf("%d", &n);
 return n;
}
```

Type 4 — Example

Scenario

A college application displays a menu to the user.
Create a function `getChoice()` that:

- takes **no arguments**
- displays the menu
- reads the user's choice
- returns the selected choice to `main()`

Menu

1. Student Details
2. Attendance
3. Marks
4. Exit

Type 4 — Solution

Solution

```
#include <stdio.h>
int getChoice(void);
int main()
{
 int choice;
 choice = getChoice();
 printf("You selected option %d", choice);
 return 0;
}
```

Type 4 — Solution

Solution

```
int getChoice(void)
{
 int choice;
 printf("===== COLLEGE MENU =====\n");
 printf("1. Student Details\n");
 printf("2. Attendance\n");
 printf("3. Marks\n");
 printf("4. Exit\n");
 printf("Enter your choice: ");
 scanf("%d", &choice);
 return choice;
}
```

Type 4 — Practice Question 1

Problem

Create a function `getNumber()` that:

- takes **no arguments**
- reads an integer from the user
- returns the number to `main()`

Example

Input: 25

Output: `Number = 25`

Type 4 — Practice Question 1 — Solution

Solution

```
#include <stdio.h>
int getNumber(void);
int main()
{
 int number;
 number = getNumber();
 printf("Number = %d", number);
 return 0;
}
```

Type 4 — Practice Question 1 — Solution

Solution

```
int getNumber(void)
{
 int n;
 printf("Enter a number: ");
 scanf("%d", &n);
 return n;
}
```

Type 4 — Practice Question 2

Problem

Create a function `getSquare()` that:

- takes **no arguments**
- reads an integer from the user
- calculates its square
- returns the square to `main()`

Example

Input: `8`

Output: `Square = 64`

Practice Question 2 — Solution

Solution

```
#include <stdio.h>
int getSquare(void);
int main()
{
 int result;
 result = getSquare();
 printf("Square = %d", result);
 return 0;
}
```

Practice Question 2 — Solution

Solution

```
int getSquare(void)
{
 int n;
 int square;
 printf("Enter a number: ");
 scanf("%d", &n);
 square = n * n;
 return square;
}
```

Type 4 — Practice Question 3

Problem

Create a function `getLargest()` that:

- takes **no arguments**
- reads two integers from the user
- finds the larger number
- returns the larger number to `main()`

Example

Enter two numbers: `25 40`

Output: `Largest = 40`

Practice Question 3 — Solution

Solution

```
#include <stdio.h>
int getLargest(void);
int main()
{
 int largest;
 largest = getLargest();
 printf("Largest = %d", largest);
 return 0;
}
```

Practice Question 3 — Solution

Solution

```
int getLargest(void)
{
 int a, b;
 printf("Enter two numbers: ");
 scanf("%d %d", &a, &b);
 if (a > b)
  return a;
 else
  return b;
}
```

Type 4 — Practice Question 4

Scenario

Create a function `getGrade()` that:

- takes **no arguments**
- reads marks from the user
- determines the grade
- returns the grade to `main()`

Grading Rules

- Marks $\geq 90 \rightarrow$ A
- Marks $\geq 75 \rightarrow$ B
- Marks $\geq 60 \rightarrow$ C
- Marks $\geq 50 \rightarrow$ D
- Marks $< 50 \rightarrow$ F

Type 4 — Practice Question 4 — Solution

Solution

```
#include <stdio.h>
char getGrade(void);
int main()
{
 char grade;
 grade = getGrade();
 printf("Grade = %c", grade);
 return 0;
}
```

Type 4 — Practice Question 4 — Solution

Solution

```
char getGrade(void)
{
 int marks;
 printf("Enter marks: ");
 scanf("%d", &marks);
 if (marks >= 90)
  return 'A';
 else if (marks >= 75)
  return 'B';
 else if (marks >= 60)
  return 'C';
 else if (marks >= 50)
  return 'D';
 else
  return 'F';
}
```

Type 4 — Practice Question 5

Scenario

Create a function `getChoice()` that:

- takes **no arguments**
- displays a calculator menu
- reads the user's choice
- returns the selected choice to `main()`

Calculator Menu

1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Exit

Type 4 — Solution

Solution

```
#include <stdio.h>
int getChoice(void);
int main()
{
 int choice;
 choice = getChoice();
 printf("You selected option %d", choice);
 return 0;
}
```

Type 4 — Solution

Solution

```
int getChoice(void)
{
 int choice;
 printf("===== CALCULATOR =====\n");
 printf("1. Addition\n");
 printf("2. Subtraction\n");
 printf("3. Multiplication\n");
 printf("4. Division\n");
 printf("5. Exit\n");
 printf("Enter your choice: ");
 scanf("%d", &choice);
 return choice;
}
```

When Should I Use Which Type of Function?

The Two Questions You Should Ask

Before choosing a function type, ask:

1. **Does the function need data from the caller?**
2. **Does the caller need a result back?**

Arguments? $\longrightarrow$ **Return Value?**

Think Before You Code

Do not choose the function type first.

Understand the problem first, then decide:

Arguments + Return Value

The 4 Types of Functions

Type	Arguments?	Return?	Basic Idea
Type 1	No	No	Task only
Type 2	Yes	No	Give data, perform task
Type 3	Yes	Yes	Give data, get result
Type 4	No	Yes	Function gets data itself, returns result

Simple Memory Trick

Arguments = Input to the function

Return value = Output from the function

When to Use Type 1?

No Arguments + No Return Value

Use Type 1 when:

- The function does not need any data from the caller.
- The function performs a fixed or predefined task.
- The result does not need to be sent back to the caller.
- The function can directly display or perform the required action.

Typical Examples

- Display a welcome message
- Display a menu
- Print a fixed pattern
- Display program instructions

Think:

Just perform the task

When to Use Type 2?

Arguments + No Return Value

Use Type 2 when:

- The function needs data from the caller.
- The function performs some processing using that data.
- The final result can be displayed or handled inside the function.
- The caller does **not need the result back**.

Examples

- `displaySquare(6)`
- `calculateBill(units)`
- `classifyNumber(number)`

Give Data → Function Performs Task

When to Use Type 3?

Arguments + Return Value

Use Type 3 when:

- The function needs data from the caller.
- The function performs some calculation or decision.
- The result is needed by the caller.
- The returned value may be stored, displayed, compared, or used in another calculation.

Examples

- `add(10,20)` → returns 30
- `largest(a,b)` → returns larger value
- `factorial(n)` → returns factorial
- `findGrade(marks)` → returns grade

Give Data → Process → Get Result

When to Use Type 4?

No Arguments + Return Value

Use Type 4 when:

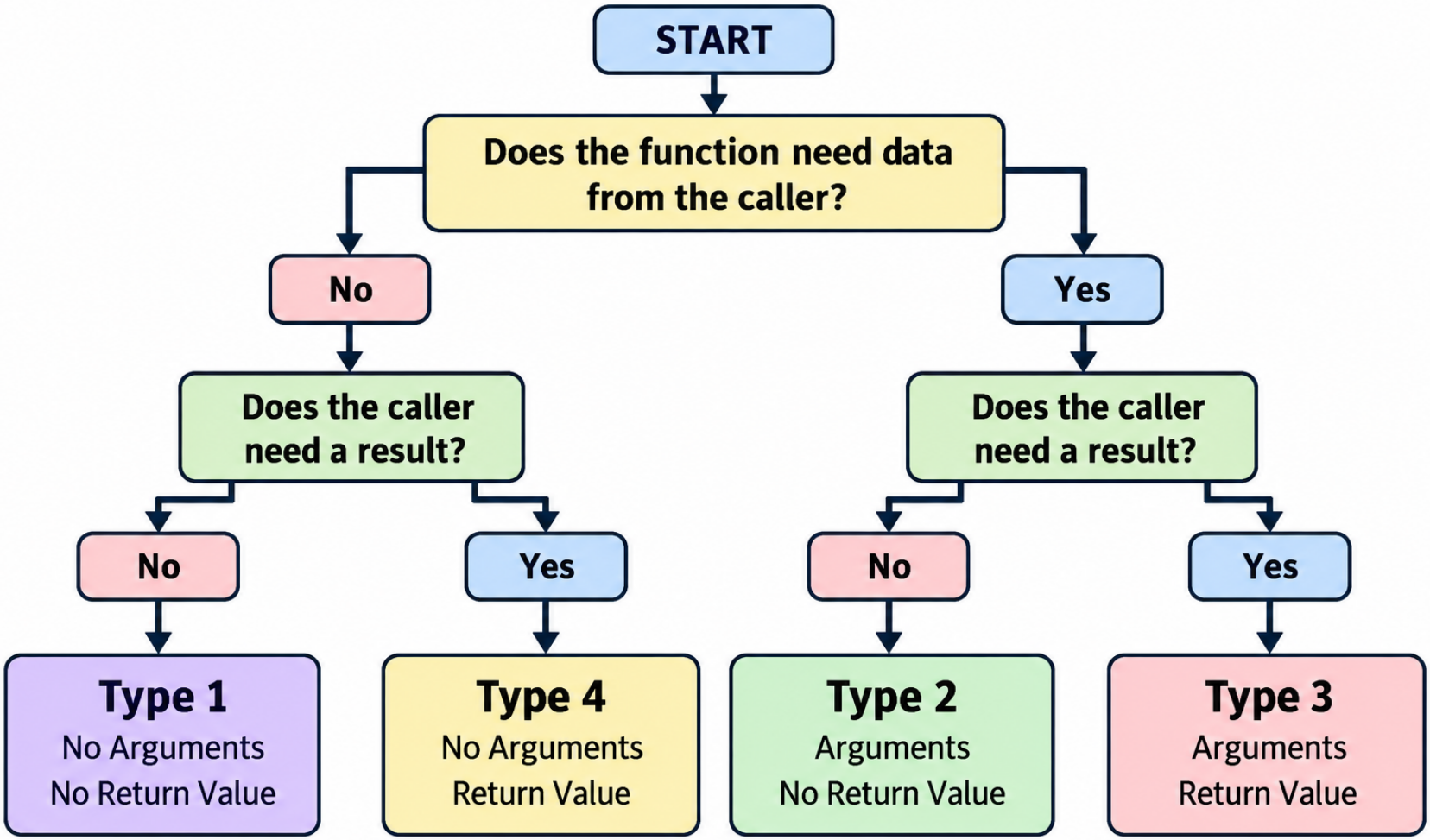
- The function does not need data from the caller.
- The function can obtain or generate the required data internally.
- The function produces a result.
- The caller needs that result.

Examples

- `getNumber()` → reads and returns a number
- `generateNumber()` → generates and returns a number
- `getChoice()` → gets and returns user's choice

Function Gets Data → Process → Return Result

How Do I Choose the Function Type?



The Same Task Can Be Designed Differently

Consider:

Calculate the square of a number

- **Type 1:** No input, no result
  - Function works with a fixed number.
- **Type 2:** Give number, function displays square
  - `displaySquare(6);`
- **Type 3:** Give number, function returns square
  - `result = square(6);`
- **Type 4:** Function gets number itself and returns square
  - `result = getSquare();`

Type 2 vs Type 3 — The Important Difference

Both functions can receive arguments.
The difference is whether the function **returns a value** or not.

Type 2: Arguments + No Return Value

- The function receives data from the caller.
- The function performs a task using that data.
- The result is displayed/used inside the function.
- **No value is returned to the caller.**

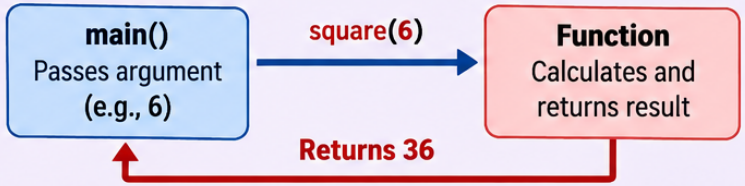
```c
#include <stdio.h>
void displaySquare(int n)
{
   int square = n * n;
   printf("Square = %d", square);
}
```



Type 3: Arguments + Return Value

- The function receives data from the caller.
- The function performs a calculation/decision.
- The result is **returned to the caller.**
- The caller can store, display, compare or use the result.

```c
#include <stdio.h>
int square(int n)
{
    return n * n;
}
```





Ask Yourself:
"Do I need the result somewhere else?"

If YES → **Return a value (Type 3)**
If NO → **No return value is enough (Type 2)**